Practice Questions on Linear Search and Binary Search

Linear Search

Linear search is a simple searching algorithm that checks each element in an array sequentially until the target element is found or the end of the array is reached.

Time Complexity: O(n)
Space Complexity: O(1)

Basic Linear Search Implementation

```java
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i; // Return index of first occurrence
        }
    }
    return -1; // Element not found
}
```

First Occurrence in Linear Search

```java
public static int firstOccurrence(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i; // Returns first occurrence index
        }
    }
    return -1;
}
```

Example:

```java
int[] arr = {2, 5, 3, 7, 3, 8, 3};
int result = firstOccurrence(arr, 3); // Returns 2 (first occurrence of 3)
```

Last Occurrence in Linear Search

```java
public static int lastOccurrence(int[] arr, int target) {
    int lastIndex = -1;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            lastIndex = i; // Update last found index
        }
    }
    return lastIndex;
}
```

Example:

```java
int[] arr = {2, 5, 3, 7, 3, 8, 3};
int result = lastOccurrence(arr, 3); // Returns 6 (last occurrence of 3)
```

Binary Search

Binary search is an efficient searching algorithm that works on sorted arrays by repeatedly dividing the search interval in half.

Time Complexity: O(log n)
Space Complexity: O(1) for iterative, O(log n) for recursive

Basic Binary Search Implementation

```java
public static int binarySearch(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

First Occurrence in Binary Search

```java
public static int firstOccurrenceBinary(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            result = mid;
            right = mid - 1; // Continue searching in left half
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return result;
}
```

Example:

```java
int[] arr = {1, 2, 3, 3, 3, 4, 5};
int result = firstOccurrenceBinary(arr, 3); // Returns 2 (first occurrence of 3)
```

Last Occurrence in Binary Search

```java
public static int lastOccurrenceBinary(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            result = mid;
            left = mid + 1; // Continue searching in right half
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return result;
}
```

Example:

```java
int[] arr = {1, 2, 3, 3, 3, 4, 5};
int result = lastOccurrenceBinary(arr, 3); // Returns 4 (last occurrence of 3)
```

Searching in Infinite Sorted Array

When dealing with an infinite sorted array, we cannot use the traditional binary search approach since we don't know the array size. We use an exponential search approach to find the range first.

```java
public static int searchInInfiniteArray(int[] arr, int target) {
    // Find the range for binary search
    int left = 0;
    int right = 1;

    // Exponentially increase the range
    while (arr[right] < target) {
        left = right;
        right = right * 2;
    }

    // Perform binary search in the found range
    return binarySearchRange(arr, target, left, right);
}

public static int binarySearchRange(int[] arr, int target, int left, int right) {
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

Example:

```java
int[] infiniteArr = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, ...};
int result = searchInInfiniteArray(infiniteArr, 13); // Returns index of 13
```

Search in Sorted Rotated Array

A rotated sorted array is a sorted array that has been rotated at some pivot point. We need to identify which part is sorted and search accordingly.

```java
public static int searchInRotatedArray(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        }

        // Check if left half is sorted
        if (arr[left] <= arr[mid]) {
            // Target is in left sorted half
            if (target >= arr[left] && target < arr[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }
        // Right half is sorted
        else {
            // Target is in right sorted half
            if (target > arr[mid] && target <= arr[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
    return -1;
}
```

Example:

```java
int[] rotatedArr = {4, 5, 6, 7, 0, 1, 2};
int result = searchInRotatedArray(rotatedArr, 0); // Returns 4 (index of 0)
```

Finding Rotation Point

```java
public static int findRotationPoint(int[] arr) {
    int left = 0;
    int right = arr.length - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] > arr[right]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return left; // Index of minimum element (rotation point)
}
```

Quiz Questions

1. What is the time complexity of linear search in the worst case? a) O(1) b) O(log n) c) O(n) d) O(n^2)

Answer: c) O(n)

2. In binary search, what condition must the array satisfy? a) Array must be unsorted b) Array must be sorted c) Array must contain unique elements d) Array must be of even length

Answer: b) Array must be sorted

3. When searching for the first occurrence of an element in a sorted array using binary search, after finding the target element, which direction should we continue searching? a) Right half b) Left half c) Both halves d) Stop searching

Answer: b) Left half

4. In a rotated sorted array [4, 5, 6, 7, 0, 1, 2], what is the rotation point (minimum element index)? a) 0 b) 3 c) 4 d) 6

Answer: c) 4

5. For searching in an infinite sorted array, what approach is used to find the search range? a) Linear expansion b) Exponential expansion c) Random selection d) Divide by 2

Answer: b) Exponential expansion

Practice Coding Questions

1. Array Rotation and Search Write a Java method to find the number of times a sorted array has been rotated. For example, if the original array was [1, 2, 3, 4, 5] and the rotated array is [3, 4, 5, 1, 2], it has been rotated 3 times.

2. Search in 2D Matrix Given a 2D matrix where each row and each column is sorted in ascending order, write an efficient algorithm to search for a target value. Return true if the target is found, false otherwise.

3. Find Peak Element A peak element in an array is an element that is greater than its neighbors. Write a method using binary search to find any peak element in O(log n) time complexity. The array may contain duplicate elements.

4. Search Insert Position Given a sorted array and a target value, return the index where the target should be inserted to maintain the sorted order. If the target already exists, return its index. Use binary search approach.

5. Count Occurrences in Sorted Array Write an efficient method to count the total number of occurrences of a given element in a sorted array. The method should run in O(log n) time complexity.